

The Adversarial Thinking Checklist

Red-team your own model before the world does.

One-line mental model: almost every attack is an optimiser searching for an input that breaks your model. So think like that optimiser, then close the gap it finds.

Quick start (15 minutes): Do steps 1-3 right now, on paper, for one system you own. Write the intended behaviour in one sentence. List three ways it could fail. Write the smallest input that would test each one. That's the whole discipline - the rest of this page makes it rigorous.

A. THE SIX-STEP LOOP - DO THIS, IN ORDER, ON ANY SYSTEM

- 1 **State intended behaviour.** What must the system do, for whom, and what counts as failure? Write it down before you test anything.
- 2 **Enumerate failure & threat modes.** List concrete ways it could go off-distribution - by accident, by an adversary, or by drift.
- 3 **Design probes.** For each mode, craft the smallest input that would expose it. Prefer the attacker's easiest move, not the fanciest one.
- 4 **Measure the gap.** Run the probes. Record an attack success rate - a number - not a vibe. Compare against step 1.
- 5 **Mitigate.** Apply the cheapest defence that closes the largest gap: input/output mediation, adversarial training, least privilege, human sign-off for high-risk actions.
- 6 **Monitor.** Failures come back after deployment. Log, re-probe on a schedule, watch for distribution drift.

B. FAILURE-MODE TRIGGER QUESTIONS

- ☐ **Distribution shift** - does it hold on inputs unlike the test set (new dialect, domain, format)?
- ☐ **Shortcut / spurious cue** - is it right for the wrong reason, using a cue that won't hold in the wild?
- ☐ **Data leakage** - did test data leak into training or the prompt?
- ☐ **Adversarial perturbation** - does a tiny, targeted change flip the output?
- ☐ **Prompt injection (direct)** - can the user override your instructions?
- ☐ **Prompt injection (indirect)** - can retrieved/third-party content hijack it?
- ☐ **Jailbreak** - does a template + search elicit disallowed behaviour?
- ☐ **Overconfidence** - is it confidently wrong, with no signal of uncertainty?
- ☐ **Evaluation illusion** - does the metric measure what matters, in the target language and context?

C. EVALUATE HONESTLY (AVOID FALSE SECURITY)

- ☐ **Attack adaptively.** No single fixed attack generalises. If an attack fails, iterate the method - don't declare victory.
- ☐ **Distrust a clean number.** Gradient masking / obfuscated gradients can lower a metric without adding real robustness.
- ☐ **Report reproducibly.** State the attack, the budget, and the success rate - not just a pass/fail.
- ☐ **Evaluate locally & in-language.** Imported benchmarks don't transfer - test in the languages and contexts you actually deploy in.

D. SAME LENS, ANY MODEL

Vision: imperceptible perturbation or adversarial patch flips a classifier

Tabular/scoring: crafted feature values game a credit or fraud model

Speech (ASR): added noise rewrites the transcript

LLM/agent: jailbreak, prompt injection, tool misuse

Recommender/ranking: poisoned interactions shift outputs

Full method, runnable notebook, and reading list ship with the session repository. All techniques cited to their original authors (Goodfellow, Madry, Papernot, Zou, Athalye/Carlini, Andriushchenko et al.; African evaluation: IrokoBench, AfroBench).